

Global Counterfactual Explainer for Graph Neural Networks

Zexi Huang*
University of California
Santa Barbara, CA, USA
zexi_huang@cs.ucsb.edu

Mert Kosan*
University of California
Santa Barbara, CA, USA
mertkosan@cs.ucsb.edu

Sourav Medya
University of Illinois
Chicago, IL, USA
medya@uic.edu

Sayan Ranu
Indian Institute of Technology
Delhi, India
sayanranu@cse.iitd.ac.in

Ambuj Singh
University of California
Santa Barbara, CA, USA
ambuj@cs.ucsb.edu

ABSTRACT

Graph neural networks (GNNs) find applications in various domains such as computational biology, natural language processing, and computer security. Owing to their popularity, there is an increasing need to explain GNN predictions since GNNs are black-box machine learning models. One way to address this is *counterfactual* reasoning where the objective is to change the GNN prediction by minimal changes in the input graph. Existing methods for counterfactual explanation of GNNs are limited to instance-specific *local* reasoning. This approach has two major limitations of not being able to offer global recourse policies and overloading human cognitive ability with too much information. In this work, we study the *global* explainability of GNNs through global counterfactual reasoning. Specifically, we want to find a *small* set of representative counterfactual graphs that explains *all* input graphs. Towards this goal, we propose GCFEXPLAINER, a novel algorithm powered by *vertex-reinforced random walks* on an *edit map* of graphs with a *greedy summary*. Extensive experiments on real graph datasets show that the global explanation from GCFEXPLAINER provides important high-level insights of the model behavior and achieves a **46.9%** gain in recourse coverage and a **9.5%** reduction in recourse cost compared to the state-of-the-art local counterfactual explainers.

KEYWORDS

Counterfactual explanation; Graph neural networks

ACM Reference Format:

Zexi Huang, Mert Kosan, Sourav Medya, Sayan Ranu, and Ambuj Singh. 2023. Global Counterfactual Explainer for Graph Neural Networks. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining (WSDM '23)*, February 27–March 3, 2023, Singapore. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

*Both authors contributed equally to this research.

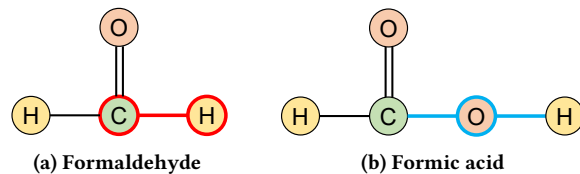


Figure 1: Formaldehyde (a) is classified by a GNN to be an undesired mutagenic molecule with its important subgraph found by factual reasoning highlighted in red. Formic acid (b) is its non-mutagenic counterfactual example obtained by removing one edge and adding one node and two edges.

1 INTRODUCTION

Graph Neural Networks (GNNs) are being used in many domains such as drug discovery [8], designing chips [18], and recommendation engines [34]. Taking the graph(s) as input, GNNs are trained to perform various downstream tasks, including node classification, graph classification, and link prediction. These downstream predictive tasks form the core of various real-world applications. For example, graph classification has been applied to predict whether a drug would exhibit the desired chemical activity [8]. Similarly, node prediction is used to predict the functionality of proteins in protein-protein interaction networks [4] and categorize users into roles on social networks [32].

Despite the impressive success of GNNs on predictive tasks, GNNs are *black-box* machine learning models. It is non-trivial to explain or reason why a particular prediction is made by a GNN. Explainability of a prediction model is important to understand its shortcomings and areas for improvement. In addition, the ability to explain a model is critical towards making it trustworthy. Owing to this limitation of GNNs, there has been significant efforts in recent times towards explanation approaches.

Existing work on explaining GNN predictions can be categorized mainly in two directions: 1) factual reasoning [15, 29, 33, 35], and 2) counterfactual reasoning [1, 2, 14, 27]. Generally speaking, the methods in the first category aim to find an important subgraph that correlates most with the underlying GNN prediction. In contrast, the methods with counterfactual reasoning attempt to identify the smallest amount of perturbation on the input graph that changes the GNN's prediction, for example, removal/addition of edges or nodes.

Compared to factual reasoning, counterfactual explainers have the additional advantage of providing the means for recourse [28].

For example, in the applications of drug discovery [8, 31], mutagenicity is an adverse property of a molecule that hampers its potential to become a marketable drug [9]. In Figure 1, formaldehyde is classified by a GNN to be mutagenic. Factual explainers can attribute the subgraph containing the carbon-hydrogen bond to the cause of mutagenicity, while counterfactual explainers provide an effective way (i.e., recourse) to turn formaldehyde into formic acid, which is non-mutagenic, by replacing a hydrogen atom with a hydroxyl.

In this work, we focus on counterfactual explanations. Our work is based on the observation that existing counterfactual explainers [15, 29, 33, 35] for graphs take a *local* perspective, generating counterfactual examples for individual input graphs. However, this approach has two key limitations:

- **Lack of global insights:** It is desirable to offer insights that generalize across a multitude of data graphs. For example, instead of providing formic acid as a counterfactual example to formaldehyde, we can summarize global recourse rules such as “Given any molecule with a carbonyl group (carbon-oxygen double bond), it needs a hydroxy to be non-mutagenic”. This focus on global counterfactual explanation promises to provide higher-level insights that are complementary to those obtained from local counterfactual explanations.
- **Information overload:** The primary motivation behind counterfactual analysis is to provide human-intelligible explanations. With this objective, consider real-world graph datasets that routinely contain thousands to millions of graphs. Owing to instance-specific counterfactual explanations, the number of counterfactual graphs grows linearly with the graph dataset size. Consequently, the sheer volume of counterfactual graphs overloads human cognitive ability to process this information. Hence, the initial motivation of providing human-intelligible insights is lost if one does not obtain a holistic view of the counterfactual graphs.

Contributions: In this paper, we study the problem of model-agnostic, *global* counterfactual explanations of GNNs for graph classification. More specifically, given a graph dataset, our goal is to counterfactually explain the largest number of input graphs with a small number of counterfactuals. As we will establish later in our experiments, this formulation naturally forces us to remove redundancy from instance-specific counterfactual explanations and hence has higher information density. Algorithmically, the proposed problem introduces new challenges. We theoretically establish that the proposed problem is NP-hard. Furthermore, the space of all possible counterfactual graphs itself is exponential. Our work overcomes these challenges and makes the following contributions:

- **Novel formulation:** We formulate the novel problem of *global* counterfactual reasoning/explanation of GNNs for graph classification. In contrast to existing works on counterfactual reasoning that only generate instance-specific examples, we provide an explanation on the global behavior of the model.
- **Algorithm design:** While the problem is NP-hard, we propose an algorithm called GCFEXPLAINER that organizes the exponential search space as an *edit map*. On this edit map, we perform *vertex-reinforced random walks* that generate diverse, representative counterfactual candidates which are *greedily summarized* as the global explanation.

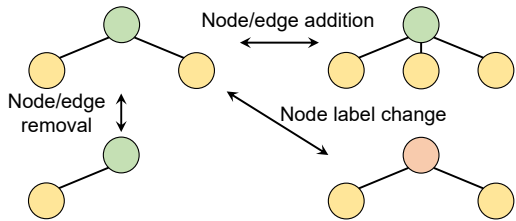


Figure 2: Edits between graphs.

- **Experiments:** We conduct extensive experiments on real-world datasets to validate the effectiveness of the proposed method. Results show that GCFEXPLAINER not only provides important high-level insights on the model behavior but also outperforms state-of-the-art baselines related to counterfactual reasoning in various recourse quality metrics.

2 GLOBAL COUNTERFACTUAL EXPLANATIONS

This section introduces the global counterfactual explanation problem for graph classification. We start with the background on local counterfactual reasoning. Then, we propose a representation of the global recourse rule that provides a high-level counterfactual understanding of the classifier behavior. Finally, we introduce quality measures for recourse rules and formally define the global counterfactual explanation problem.

2.1 Local Counterfactual

Consider a graph $G = (V, E)$ where V and E are the sets of nodes and edges respectively. A (binary) graph classifier (e.g., a GNN) ϕ classifies G into either the undesired class ($\phi(G) = 0$) or the desired one ($\phi(G) = 1$). An explanation of ϕ seeks to answer how these predictions are made. Those based on factual reasoning analyze what properties G possesses to be classified in the current class while those based on counterfactual reasoning find what properties G needs to be assigned to the opposite class.

Existing counterfactual explanation approaches take a local perspective. Specifically, for each input graph G , they find a **counterfactual** (graph) C that is somewhat similar to G but is assigned to a different class. Without loss of generality, let G belong to the undesired class, i.e., $\phi(G) = 0$, then the counterfactual C satisfies $\phi(C) = 1$. The similarity between C and G is quantified by a predefined distance metric d , for example, the number of added/removed edges [2, 14].

In our work, we consider the graph edit distance (GED) [24], a more general distance measure, as the distance function to account for other types of changes. Specifically, $\text{GED}(G_1, G_2)$ counts the minimum number of “edits” to convert G_1 to G_2 . An “edit” can be the addition or removal of edges and nodes, or change of node labels (see Figure 2). Moreover, to account for graphs of different sizes, we normalize the GED by the sizes of graphs: $\widehat{\text{GED}}(G_1, G_2) = \text{GED}(G_1, G_2) / (|V_1| + |V_2| + |E_1| + |E_2|)$. Nonetheless, our method can be applied with other graph distance metrics, such as those based on graph kernels (e.g., RW [3], NSPDG [5], WL [26]).

The distance function measures the quality of the counterfactual found by the explanation model. Ideally, the counterfactual C should

be very close to the input graph G while belonging to a different class. Formally, we define the counterfactuals that are within a certain distance θ from the input graph as **close counterfactuals**.

DEFINITION 1 (CLOSE COUNTERFACTUAL). *Given the GNN classifier ϕ , distance parameter θ , and an input graph G with undesired outcome i.e., $\phi(G) = 0$; a counterfactual graph, C , is a close counterfactual of G when $\phi(C) = 1$ and $d(G, C) \leq \theta$.*

While the (close) counterfactual C found by existing approaches provides an explanation of the classifier behavior for the corresponding input graph G , it is hard to generalize to understand the global behavior. Next, we introduce the global recourse rule that provides a high-level summary of the classifier behavior across different input graphs.

2.2 Global Recourse Representation

The global counterfactual explanation requires a global recourse rule r . Specifically, for any input graph G with $\phi(G) = 0$, r provides a (close) counterfactual (i.e., a recourse) for G : $\phi(r(G)) = 1$. While both a recourse rule and a local counterfactual explainer find a counterfactual given an input graph, their goals are different. The goal of the local counterfactual explainer is to find the best (closest) counterfactual possible for each input graph, and therefore, r can be very complicated, e.g., in the form of an optimization algorithm [2, 27]. On the other hand, a recourse rule aims to provide an explanation of the classifier’s global behavior, which requires a simpler form that is understandable for domain experts without prior knowledge of deep learning on graphs.

Existing global recourse rules for classifiers with feature vectors as input take the form of short decision trees [22]. However, this is hard to be generalized to graph data with rich structure information. Instead, we propose the representation of a global recourse rule for a graph classifier to be a collection of counterfactual graphs $\mathbb{C}$ that is *diverse* and *representative* enough to capture its global behavior. This representation does not require any additional knowledge for domain experts to understand and draw insights from, similar to the local counterfactual examples. It is also easy to find the local counterfactual for a given input graph G based on $\mathbb{C}$ by nominating the closest graph in $\mathbb{C}$: $r(G) = \arg \min_{C \in \mathbb{C}} d(G, C)$.

2.3 Quantifying Recourse Quality

Given a set of n input graphs $\mathbb{G}$ and a graph classifier ϕ , we want to compare the quality of different recourse representations $\mathbb{C}$. Similar to the quality metrics introduced for recourse for vector data [22], we aim to account for the following factors:

- (1) **Coverage:** Like local counterfactual explainers, we want to ensure that counterfactuals found for individual input graphs are of high quality. Specifically, we introduce *recourse coverage*—the proportion of input graphs that have close counterfactuals based on $\mathbb{C}$:

$$\text{coverage}(\mathbb{C}) = |\{G \in \mathbb{G} \mid \min_{C \in \mathbb{C}} \{d(G, C)\} \leq \theta\}| / |\mathbb{G}|$$

- (2) **Cost:** Another quality metric based on local counterfactual quality is the recourse cost (i.e., the distance between the input graph and its counterfactual) across the input graphs:

$$\text{cost}(\mathbb{C}) = \text{agg} \left\{ \min_{C \in \mathbb{C}} \{d(G, C)\} \right\}_{G \in \mathbb{G}}$$

where agg is an *aggregation* function, e.g., mean or median.

- (3) **Interpretability:** Finally, the recourse rule should be easy (small) enough for human cognition. We quantify the interpretability as the size of recourse representation:

$$\text{size}(\mathbb{C}) = |\mathbb{C}|$$

2.4 Problem Formulation and Characterization

An ideal recourse representation should maximize the coverage while minimizing the cost and the size. Formally, we define the global counterfactual explanation problem as follows:

PROBLEM 1 (GLOBAL COUNTERFACTUAL EXPLANATION FOR GRAPH CLASSIFICATION (GCE)). *Given a GNN graph classifier ϕ that classifies n input graphs $\mathbb{G}$ to class 0 and a budget $k \ll n$, our goal is to find the best recourse representation $\mathbb{C}$ that maximizes the recourse coverage with size k :*

$$\max_{\mathbb{C}} \text{coverage}(\mathbb{C}) \text{ s.t. } \text{size}(\mathbb{C}) = k$$

We note that in our problem formulation only coverage and size are explicitly accounted for, whereas cost is absent. We make this design choice since cost and coverage are intrinsically opposing forces. Specifically, if we are willing to allow a high cost, coverage increases since we allow for higher individual distances between an input graph and its counterfactual. Therefore, we take the approach of binding the cost to the distance threshold θ in the coverage definition. Nonetheless, an explicit analysis of all these metrics including cost is performed to quantify recourse quality during our empirical evaluation in Section 4.

THEOREM 1 (NP-HARDNESS). *The GCE problem is NP-hard.*

PROOF. To establish NP-hardness of the proposed problem we reduce it from the classical *Maximum Coverage* problem.

DEFINITION 2 (MAXIMUM COVERAGE). *Given a budget k and a collection of subsets $\mathcal{S} = \{S_1, \dots, S_m\}$ from a universe of items $U = \{u_1, \dots, u_n\}$, find a subset $S' \subseteq \mathcal{S}$ of sets such that $|S'| \leq k$ and the number of covered elements is maximized, i.e., maximize $|\bigcup_{S_i \in S'} S_i|$.*

We show that given any instance of a maximum coverage problem $(\mathcal{S}, U)$, it can be mapped to the GCE problem. Specifically, we assume that each element u_i in U corresponds to an input undesired graph G_i . For u_i , assume a (star) graph with a center node with empty label and n nodes with empty labels except the node at i -th position that has label u_i . For S_i , assume similar graph with a center node with blank label and nodes with labels with the elements in S_i in their corresponding positions. The edit operations that are allowed are either making a node label blank or add a new label in their position and but not both. So, each S_i corresponds to a $C_i \in \mathbb{C}$ and $d(G_j, C_i) \leq \theta$ iff $u_j \in S_i$ where θ is assumed large enough. Note that with this construction, it is easy to see that an optimal solution for this instance of the GCE problem is the optimal solution for the corresponding instance of the maximum coverage problem. $\square$

Owing to NP-hardness, it is not feasible to identify the optimal solution for the GCE problem in polynomial time. In the next section, we will introduce GCFEXPLAINER, an effective and efficient heuristic that solves the GCE problem.

3 PROPOSED METHOD: GCFEXPLAINER

In this section, we propose GCFEXPLAINER, the first global counterfactual explainer for graph classification. The GCE problem requires us to find a collection of k counterfactual graphs that maximize the coverage of the input graphs. Intuitively, we want each individual counterfactual graph to be a close counterfactual to (i.e., “cover”) as many input graphs as possible. Additionally, different counterfactual graphs should be able to cover different sets of input graphs so that the overall coverage is maximized.

These intuitions motivate the design of our algorithm GCFEXPLAINER, which has three major components:

- (1) **Structuring the search space:** The search space of counterfactual graphs consists of *all* graphs that are in the same domain as the input graphs and within a distance of θ . In other words, any graph within a distance of θ from an input graph may be a potential counterfactual candidate and therefore needs to be analyzed. The number of potential graphs within θ increases exponentially with θ since the space of graph edits is combinatorial [13]. GCFEXPLAINER uses an *edit map* to organize these graphs as a meta-graph $\mathcal{G}$, where individual nodes are graphs that are created via a different number of edits from the input graphs and each edge represents a single edit.
- (2) **Vertex-reinforced random walk:** To search for good counterfactual candidates, GCFEXPLAINER leverages vertex-reinforced random walks (VRRW) [20] on the edit map $\mathcal{G}$. VRRW has the nice property of converging to a set of nodes that are both important (i.e., cover many input graphs) and diverse (i.e., non-overlapping coverage), which will form a small set of counterfactual candidates for further processing.
- (3) **Iterative computation of summary:** After obtaining good counterfactual candidates from VRRW, GCFEXPLAINER creates the final set of the counterfactual graphs (i.e., the summary) as the recourse representation by iteratively adding the best candidate based on the maximal gain of the coverage given the already added candidates.

3.1 Edit Map

The search space for counterfactual graphs in GCFEXPLAINER is organized via an edit map $\mathcal{G}$. The edit map is a meta-graph whose nodes are graphs in the same domain as the input graphs and edges connect graphs that differ by a single graph edit. As an example, each graph in Figure 2 represents a node in the edit map, and the arrows denote edges between graphs (nodes) that are one edit away. In the edit map, we only include connected graphs since real graphs of interest are often connected (e.g., molecules, proteins, etc.).

While all potential counterfactual candidates are included as its nodes, the edit map has an exponential size and it is computationally prohibitive to fully explore it. However, a key observation is that a counterfactual candidate can only be a few hops away from some input graph, otherwise, the graph distance between the counterfactual and the input graph would be too large for the counterfactual to cover it. This observation motivates our exploration of the edit map to be focused on the union of close neighborhoods of the input graphs (see Section 3.2.3). Additionally, while we cannot compute the entire edit map, it is easy to chart the close neighborhoods by iteratively performing all possible edits from the input

graphs. Next, we introduce the vertex-reinforced random walk to efficiently explore the edit map to find counterfactual candidates.

3.2 Vertex-Reinforced Random Walk

Vertex-reinforced random walk (VRRW) [20] is a time-variant random walk. Different from other more widely applied random walk processes such as the simple random walk and the PageRank [7, 12, 21], the transition probability $p(u, v)$ of VRRW from node u to node v depends not only on the edge weight $w(u, v)$ but also the *number of previous visits* in the walk to the target node v , which we denote using $N(v)$. Specifically,

$$p(u, v) \propto w(u, v)N(v) \quad (1)$$

GCFEXPLAINER applies VRRW on the edit map and produces n most frequently visited nodes in the walk as the set of counterfactual candidates $\mathbb{S}$. Next, we formalize VRRW in our setting and explain how it surfaces good counterfactual candidates for GCE.

3.2.1 Vertex-reinforcement. Our main motivation for using VRRW to explore the edit map instead of other random walk processes is that VRRW converges to a diverse and representative set of nodes [16, 19] in different regions of the edit map. In this way, the frequently visited nodes in instances of VRRW have the potential to be good counterfactual candidates and they would cover a diverse set of input graphs in the edit map. The reason behind the diversity of the highly visited nodes is the previous visit count $N(v)$ in the transition probability. Specifically, nodes with high visit counts tend to be visited more often later (“richer gets richer”), and thereby dominating all other nodes in their neighborhood. This leaves a bunch of highly visited nodes to “represent” each region of the edit map. We refer the readers to [16] for details on the mathematical basis and the theoretical correctness of this property. Moreover, as our goal is to find counterfactual candidates, we only reinforce (i.e., increase the visit counts of) graphs in the counterfactual class.

3.2.2 Importance function. While the vertex-reinforcement mechanism ensures diversity of the highly visited nodes, we still need to guide the walker to visit graphs that are good counterfactual candidates. We achieve this by assigning large edge weight $w(u, v)$ to good counterfactual candidates via an importance function $I(v)$:

$$w(u, v) = I(v) \quad (2)$$

The importance function $I(v)$ should capture the quality of a graph v as a counterfactual candidate. It has the following components:

- (1) Counterfactual probability $p_\phi(v)$. The graph classifier ϕ predicts a probability for v to be in the counterfactual class ($\phi(v) = 1$). By using it as part of the importance function, the walker is encouraged to visit regions with rich counterfactual graphs.
- (2) Individual coverage **coverage**($\{v\}$). The individual coverage of a graph v computes the proportion of input graphs that are close to v . This encourages the walker to visit graphs that cover many input graphs.
- (3) Gain of coverage **gain**($v; \mathbb{S}$). Given a graph v and the current set of counterfactual candidates $\mathbb{S}$ (i.e., the n most frequently visited nodes), we can compute the gain between the current coverage and the coverage after adding v to $\mathbb{S}$:

$$\mathbf{gain}(v; \mathbb{S}) = \mathbf{coverage}(\mathbb{S} \cup \{v\}) - \mathbf{coverage}(\mathbb{S})$$

This guides the walker to find graphs that compliment the current counterfactual candidates to cover additional input graphs. The importance function is a combination of these components:

$$I(v) = p_\phi(v)(\alpha \text{coverage}(\{v\}) + (1 - \alpha) \text{gain}(v; \mathbb{S})) \quad (3)$$

where α is a hyperparameter between 0 and 1. With the above importance function, the VRRW in GCFEXPLAINER converges to a set of diverse nodes that have high counterfactual probability and cover a large number of input graphs.

3.2.3 Dynamic teleportation. The last component of VRRW, teleportation, is to help us manage the exponential search space of the edit map. Since our goal is to find close counterfactuals to the input graphs, the walker only needs to explore the nearby regions of the input graphs. Therefore, we start the walk from the input graphs, and also at each step, let the walker teleport back (i.e. transit) to a random input graph with probability τ .

To decide which input graph to teleport to, we adopt a dynamic probability distribution based on the current counterfactual candidate set $\mathbb{S}$. Specifically, let $g(G) = |\{v \in \mathbb{S} \mid d(v, G) \leq \theta \text{ and } \phi(v) = 1\}|$ be the number of close counterfactuals in $\mathbb{S}$ covering an input graph G . Then the probability to teleport to G is

$$p_\tau(G) = \frac{\exp(-g(G))}{\sum_{G' \in \mathcal{G}} \exp(-g(G'))} \quad (4)$$

This dynamic teleportation favors input graphs that are not well covered by the current solution set and encourages the walker to explore potential counterfactuals to cover the uncovered input graphs after teleportation.

3.3 Iterative Computation of the Summary

We have applied VRRW to generate a good set of n counterfactual candidates $\mathbb{S}$. In the last step of GCFEXPLAINER, we aim to further refine the candidate set and create the final recourse representation (i.e., the summary) with k counterfactual graphs. This summarization problem is also NP-hard and we propose to build $\mathbb{C}$ in an iterative and greedy manner from $\mathbb{S}$.

Specifically, we start with an empty solution set $\mathbb{C}_0$. Then, for each iteration t , we add the graph v to $\mathbb{C}_t$ with the maximal gain of coverage $\text{gain}(v; \mathbb{C}_t)$. This is repeated k times to get the final recourse representation $\mathbb{C}$ with k graphs. It is easy to show that the summarization problem is submodular and therefore, our greedy algorithm provides $(1 - 1/e)$ -approximation.

Notice that the greedy algorithm can also be applied to the local counterfactuals found by existing methods to generate a GCE solution. Here, we highlight three advantages of GCFEXPLAINER:

- (1) Existing local counterfactual explainers [1, 2, 14, 27] are only able to generate counterfactuals based on one type of graph edits—edge removal, while GCFEXPLAINER incorporates all types of edits to include a richer set of counterfactual candidates.
- (2) The set of counterfactual candidates from GCFEXPLAINER is generated with the GCE objective in mind, while the local counterfactuals from existing methods are optimized for individual input graphs. Therefore, they may not be good candidates to capture the global behavior of the classifier.

Algorithm 1 GCFEXPLAINER

```

1:  $G \leftarrow$  random input graph,  $N(G) \leftarrow 1$ ,  $\mathbb{S} = \{G\}$ 
2: for  $i \in 1 : M$  do
3:   Let  $\epsilon \sim \text{Bernoulli}(\tau)$ 
4:   if  $\epsilon = 0$  then
5:     for  $v \in \text{Neighbors}(G)$  do
6:       Compute  $I(v)$  based on Equation 3
7:       Compute  $p(G, v)$  based on Equation 1
8:        $v \leftarrow$  random neighbor of  $G$  based on  $p(G, v)$ 
9:   else
10:     $v \leftarrow$  random input graph based on Equation 4
11:   if  $\phi(v) = 1$  then
12:     if  $v \in \mathbb{S}$  then
13:        $N(v) \leftarrow N(v) + 1$ 
14:     else
15:        $\mathbb{S} \leftarrow \mathbb{S} + \{v\}$ ,  $N(v) \leftarrow 1$ 
16:    $G \leftarrow v$ 
17:  $\mathbb{S} \leftarrow$  top  $n$  frequently visited counterfactuals in  $\mathbb{S}$ 
18:  $\mathbb{C} \leftarrow \emptyset$ 
19: for  $t \in 1 : k$  do
20:    $v \leftarrow \arg \max_{v \in \mathbb{S}} \text{gain}(v; \mathbb{C})$ 
21:    $\mathbb{C} \leftarrow \mathbb{C} + \{v\}$ 
22: return  $\mathbb{C}$ 

```

- (3) It is easy to incorporate domain constraints (e.g., the valence of chemical bonds) into GCFEXPLAINER by pruning the neighborhood of the edit map, while existing methods based on optimization require nontrivial efforts to customize.

We will empirically demonstrate the superiority of GCFEXPLAINER to this two-stage approach with state-of-the-art local counterfactual explanation methods in our experiments in Section 4.2.

Pseudocode and complexity: The pseudocode of GCFEXPLAINER is presented in Algorithm 1. Line 1-16 summarizes the VRRW component of GCFEXPLAINER. Specifically, Line 3-10 determines the next graph to visit based on VRRW transition probabilities and dynamic teleportation, and Line 11-16 update the visit counts and the set of counterfactual candidates. The iterative computation of the counterfactual summary is described in Line 17-21. The overall complexity of GCFEXPLAINER is $O(Mhn + kn)$, where M is the number of iterations for the VRRW, h is the average node degree in the meta-graph, n is the number of input graphs, and k is the size of the global counterfactual representation. In practice, we store the computed transition probabilities with a space-saving algorithm [17] to improve the running time of GCFEXPLAINER.

4 EXPERIMENTS

We provide empirical results for the proposed GCFExplainer along with baselines on commonly used graph classification datasets. The implementation is anonymously available¹.

4.1 Experimental Settings

4.1.1 Datasets. We use four different real-world datasets for graph classification benchmark with their statistics in Table 1. Specifically, NCI1 [30], Mutagenicity [9, 23], and AIDS [23] are collections of

¹<https://anonymous.4open.science/r/gcf-975A/>

molecules with nodes representing different atoms and edges representing chemical bonds between them. The molecules are classified by whether they are anticancer, mutagenic, and active against HIV, respectively. Proteins [4, 6] is a collection of proteins classified into enzymes and non-enzymes, with nodes representing secondary structure elements and edges representing structural proximity. For all datasets, we filter out graphs containing rare nodes with label frequencies smaller than 50.

Table 1: The statistics of the datasets.

	NCI1	Mutagenicity	AIDS	Proteins
#Graphs	3978	4308	1837	1113
#Nodes	118714	130719	28905	43471
#Edges	128663	132707	29985	81044
#Node Labels	10	10	9	3

4.1.2 Graph classifier. We follow [29] and train a GNN with 3 convolution layers [11] of embedding dimension 20, a max pooling layer, and a fully connected layer for classification. The model is trained with the Adam optimizer [10] and a learning rate of 0.001 for 1000 epochs. The datasets are split into 80%/10%/10% for training/validation/testing with the model accuracy shown in Table 3.

4.1.3 Baselines. To the best of our knowledge, GCFEXPLAINER is the first global counterfactual explainer. To validate its effectiveness, we compare it against state-of-the-art local counterfactual explainers combined with a greedy summarization algorithm as described in Section 3.3. The following local counterfactual generation methods are included in our experiments.

- **GROUND-TRUTH:** Using graphs belonging to the desired class from the original dataset as local counterfactuals.
- **RCEXPLAINER [2]:** Local counterfactual explainer based on the modeling of implicit decision regions of GNNs.
- **CFF [27]:** Local counterfactual explainer based on joint modeling of factual and counterfactual reasoning.

4.1.4 Explainer settings. We use a distance threshold θ of 0.05 for training all explainers. For GCFEXPLAINER, we set the teleportation probability $\tau = 0.1$ and tune α , the weight between individual coverage and gain of coverage, from $\{0, 0.5, 1\}$. A sensitivity analysis is presented in Section 4.6. The number of VRRW iterations M is set to 50000, which is enough for convergence as shown in Section 4.5. For baselines, we tune their hyperparameters to achieve the best local counterfactual rates while maintaining an average distance to input graphs that is smaller than the distance threshold θ .

4.2 Recourse Quality

We start by comparing the recourse quality between GCFEXPLAINER and baselines. Table 2 shows the recourse coverage with $\theta = 0.1$ and median recourse cost of the top 10 counterfactual graphs (i.e., $k = 10$). We first notice that the two state-of-the-art local counterfactual explainers have similar performance as GROUND-TRUTH, consistent with our claim that local counterfactual examples from existing methods are not good candidates for a global explanation. The proposed GCFEXPLAINER, on the other hand, achieves significantly better performance for global recourse quality. Compared to the

best baseline, RCEXPLAINER, GCFEXPLAINER realizes a **46.9%** gain in recourse coverage and a **9.5%** reduction in recourse cost.

Next, we show the recourse coverage and cost for different sizes of counterfactual summary in Figure 3. As expected, adding more graphs to the recourse representation increases recourse coverage while decreasing recourse cost, at the cost of interpretability. And GCFEXPLAINER consistently outperforms the baselines across different sizes.

We also compare the recourse coverage based on different distance thresholds to demonstrate whether the methods can be generalized with different values of θ . The results are shown in Figure 4. While coverage increases for all methods as the threshold increases, GCFEXPLAINER maintains a constant edge over the baselines.

4.3 Global Counterfactual Insight

We have demonstrated the superiority of GCFEXPLAINER based on various quality metrics for global recourse. Here, we show how GCFEXPLAINER provides global insights compared to local counterfactual examples. Figure 5 illustrates (a) four input undesired graphs with a similar structure, (b) corresponding local counterfactual examples (from RCEXPLAINER and CFF), and (c) the representative global counterfactual graph from GCFEXPLAINER covering the input graphs, for the AIDS dataset. Our goal is to understand why the input graphs are inactive against AIDS (undesired) and how to make them active with minimal changes.

The local counterfactuals in (b) attribute the classification results to different edges in individual graphs (shown as red dotted lines) and recommend their removal to make input graphs active against HIV. Note that while only two edits are proposed for each individual graph, they appear at different locations, which are hard to generalize for a global view of the model behavior. In contrast, the global counterfactual graph from GCFEXPLAINER presents a high-level recourse rule. Specifically, the carbon atom with the carbon-oxygen bond is connected to *two* other carbon atoms in the input graphs, making them ketones (with a C=O bond) or ethers (with a C-O bond). On the other hand, the global counterfactual graph highlights a different functional group, aldehyde (shown in blue), to be the key for combating AIDS. In aldehydes, the carbon atom with a carbon-oxygen bond is only connected to *one* other carbon atom, leading to different chemical properties compared to ketones and ethers. Indeed, aldehydes have been shown to be effective HIV protease inhibitors [25].

Finally, this case study also demonstrates that counterfactual candidates found by GCFEXPLAINER are better for global explanation than local counterfactuals. We note that while the graph edit distance between the local counterfactuals to their corresponding input graphs is only 2, they do not cover other similarly structured input graphs (with distance > 5). Meanwhile, our global counterfactual graph covers all input graphs (with distance ≤ 4).

4.4 Ablation Study

We then conduct an ablation study to investigate the effectiveness of GCFEXPLAINER components. We consider three alternatives:

- GCFEXPLAINER-NVR: no vertex-reinforcement ($N(v) = 1$)
- GCFEXPLAINER-NIF: no importance function ($I(v) = 1$)
- GCFEXPLAINER-NDT: no dynamic teleportation ($p_r(G) = 1/|\mathcal{G}|$)

Table 2: Recourse coverage ($\theta = 0.1$) and median recourse cost comparison between GCFEXPLAINER and baselines for a 10-graph global explanation. GCFEXPLAINER consistently and significantly outperforms all baselines across different datasets.

	NCI1		Mutagenicity		AIDS		Proteins	
	Coverage	Cost	Coverage	Cost	Coverage	Cost	Coverage	Cost
GROUND-TRUTH	16.54%	<u>0.1326</u>	28.96%	<u>0.1275</u>	0.41%	0.2012	8.47%	<u>0.2155</u>
RCEXPLAINER	15.22%	0.1370	31.99%	0.1290	8.96%	0.1531	8.74%	0.2283
CFF	<u>17.61%</u>	0.1331	30.43%	0.1327	3.39%	0.1669	3.83%	0.2557
GCFEXPLAINER	27.85%	0.1281	37.08%	0.1135	14.66%	0.1516	10.93%	0.1856

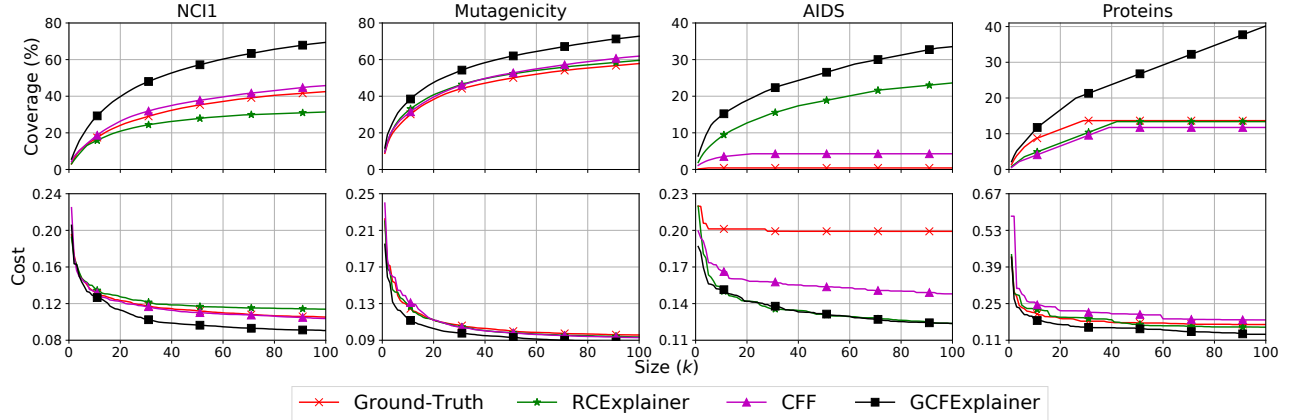


Figure 3: Coverage and cost performance comparison between GCFEXPLAINER and baselines based on different counterfactual summary sizes. GCFEXPLAINER consistently outperforms the baselines across different sizes.

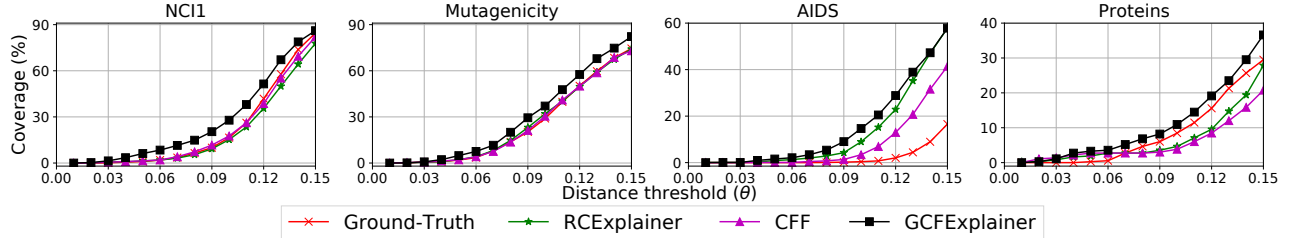


Figure 4: Recourse coverage comparison between GCFEXPLAINER and baselines based on different distance threshold values (θ). GCFEXPLAINER consistently outperforms the baselines across different θ .

Table 3: Accuracy of the GNN graph classifier.

	NCI1	Mutagenicity	AIDS	Proteins
Training	0.8439	0.8825	0.9980	0.7800
Validation	0.8161	0.8302	0.9727	0.8198
Testing	0.7809	0.8000	0.9781	0.7297

The coverage results are shown in Table 4. We observe decreased performance when any of GCFEXPLAINER components is absent.

Table 4: Ablation study results based on recourse coverage.

	NCI1	Mutagenicity	AIDS	Proteins
GCFEXPLAINER-NVR	24.56%	35.44%	11.33%	8.56%
GCFEXPLAINER-NIF	13.29%	29.16%	4.54%	6.83%
GCFEXPLAINER-NDT	27.34%	36.35%	14.05%	9.28%
GCFEXPLAINER	27.85%	37.08%	14.66%	10.93%

4.5 Convergence Analysis

In this subsection, we show the empirical convergence of VRRW based on the mutagenicity dataset in Figure 6. We observe that the coverage performance for different summary sizes starts to converge after 15000 iterations and fully converges after 50000 iterations, which is the number we applied in our experiments.

4.6 Sensitivity Analysis

The only hyperparameter of GCFEXPLAINER we tune is α in Equation 3 that weights the individual coverage and gain of coverage for the importance function. Table 5 shows the recourse coverage based on different α values. While GCFEXPLAINER outperforms baselines with all different α , we observe that individual coverage works better for NCI1 and gain of cumulative coverage works better for other datasets.

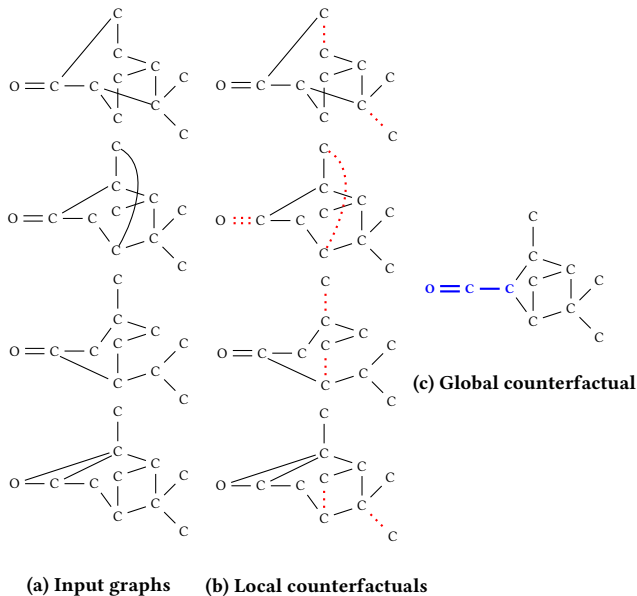


Figure 5: Illustration of global and local counterfactual explanations for the AIDS dataset. The global counterfactual graph (c) presents a high-level recourse rule—changing ketones and ethers into aldehydes (shown in blue)—to combat HIV, while the edge removals (shown in red) recommended by local counterfactual examples (b) are hard to generalize.

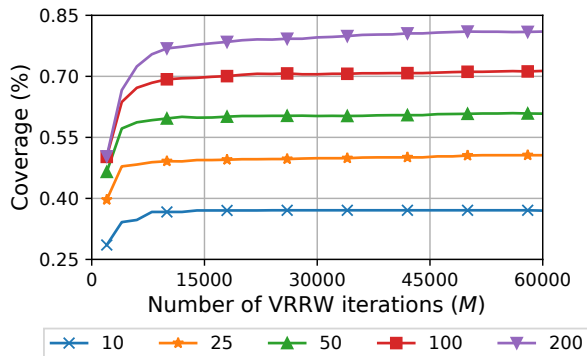


Figure 6: Convergence of VRRW for the mutagenicity dataset based on recourse coverage with different summary sizes. VRRW fully converges after $M = 50000$ iterations.

Table 5: Sensitivity analysis on α , the weight between individual coverage and gain of coverage in the importance function.

	NCI1	Mutagenicity	AIDS	Proteins
$\alpha = 0.0$	27.85%	36.87%	12.83%	10.11%
$\alpha = 0.5$	27.50%	36.59%	14.66%	10.38%
$\alpha = 1.0$	22.27%	37.08%	13.99%	10.93%

4.7 Running Time

Table 6 summarizes the running times of generating counterfactual candidates based on different methods. GCFEXPLAINER has a competitive running time albeit exploring more counterfactual graphs in the process. We also include results for GCFEXPLAINER-S which samples a maximum of 10000 neighbors for computing the importance at each step. It achieves better running time at a negligible cost of 3.3% performance loss on average. Finally, summarizing the counterfactual candidates takes less than a second for all methods.

Table 6: Counterfactual candidates generation time comparison. GCFEXPLAINER (-S) has competitive running time albeit exploring more counterfactual graphs.

	NCI1	Mutagenicity	AIDS	Proteins
RCEXPLAINER	30454	52549	29047	8444
CFF	22794	31749	21296	6412
GCFEXPLAINER	<u>19817</u>	<u>24006</u>	<u>2615</u>	19246
GCFEXPLAINER-S	19365	18798	2539	<u>7429</u>

5 RELATED WORK

Explanations for Graph Neural Networks. There is much research [15, 29, 33, 35] on explaining the mechanism of graph neural networks (GNNs). The first proposed method, GNNExplainer [33], finds a subgraph and sub-features which explain the prediction by maximizing the mutual information between the original prediction and the prediction based on the subgraph and sub-features. Later, PGExplainer [15] provides an inductive framework for generating explanations for GNNs in terms of weighted edge sets. The method extracts node embeddings from the last layer of the GNN, and then learns to map node embedding pairs to the probability of edge existence in the explanatory subgraph. A different approach for the same problem, PGMEExplainer [29] aims to learn a Bayesian network from a graph so that it can be explainable via a probabilistic model. The model learns new predictions from perturbed node features, performs variable selection using Markov blanket of variables, and then produces a Bayesian network via structure learning. In [35], the authors propose a method finding model-level explanations on trained GNNs over a graph classification task. They formulate a graph generation module using reinforcement learning which predicts the next edge to be added to the graph. These explanation methods focus on *factual* reasoning while the goal of our work is to provide a *global counterfactual* explanation for GNNs.

Counterfactual Explanations. Recently, there are several attempts to have explanations of graph neural networks (GNNs) via counterfactual reasoning [1, 2, 14, 27]. One of the earlier methods, CF-GNNExplainer [14], provides counterfactual explanations for node classification based on GNNs. The counterfactual explanation is given as a perturbed adjacency matrix that leads to the flipping of prediction for a node. It parameterizes the perturbation with a real-value mask of the original adjacency matrix, and applies the close-form formula of GCN on the perturbed matrix to generate the perturbed prediction. On the other hand, RCEExplainer [2] aims to find a subset of edges such that removing them changes the prediction of the remaining graph, while the subset is robust against the noise in the graph. The robustness is achieved via modeling

the implicit decision regions based on graph embeddings extracted from the last layer of the GNN. In [1], the authors investigate counterfactual explanations for a more specific class of graphs—the brain networks—that share the same set of nodes (Regions of Interest). They develop heuristic algorithms to find counterfactuals by greedily adding or removing edges for each input graph. More recently, in [27], the authors argue that a good explanation for GNNs should consider both factual and counterfactual reasoning. They parameterize the explanation with an adjacency mask and a feature mask, which effectively allows them to generate an explanatory subgraph with potentially changed node labels. The model is transductive and the two masks are optimized independently for each input graph. Counterfactual reasoning has also been applied for the link prediction task [36]. All the above methods produce *local* counterfactual examples where our work aims to provide a *global* explanation in terms of a summary of representative counterfactual graphs.

6 CONCLUSION

We have proposed GCFEXPLAINER, the first global counterfactual explainer for graph classification. Compared to local explainers, GCFEXPLAINER provides a high-level picture of the model behavior and effective global recourse rules. We hope that our work will not only deepen our understanding of graph neural networks but also build a bridge for experts from other domains to leverage deep learning models for high-stakes decision-making.

REFERENCES

- [1] Carlo Abrate and Francesco Bonchi. 2021. Counterfactual graphs for explainable classification of brain networks. In *SIGKDD*.
- [2] Mohit Bajaj, Lingyang Chu, Zi Yu Xue, Jian Pei, Lanjun Wang, Peter Cho-Ho Lam, and Yong Zhang. 2021. Robust Counterfactual Explanations on Graph Neural Networks. In *NeurIPS*.
- [3] Karsten Borgwardt, Nicol Schraudolph, and SVN Vishwanathan. 2006. Fast computation of graph kernels. In *NeurIPS*.
- [4] Karsten M Borgwardt, Cheng Soon Ong, Stefan Schöner, SVN Vishwanathan, Alex J Smola, and Hans-Peter Kriegel. 2005. Protein function prediction via graph kernels. *Bioinformatics* 21, suppl_1 (2005), i47–i56.
- [5] Fabrizio Costa and Kurt De Grave. 2010. Fast neighborhood subgraph pairwise distance kernel. In *ICML*.
- [6] Paul D Dobson and Andrew J Doig. 2003. Distinguishing enzyme structures from non-enzymes without alignments. *Journal of molecular biology* 330, 4 (2003), 771–783.
- [7] Zexi Huang, Arlei Silva, and Ambuj Singh. 2021. A broader picture of random-walk based graph embedding. In *SIGKDD*.
- [8] Mingjian Jiang, Zhen Li, Shugang Zhang, Shuang Wang, Xiaofeng Wang, Qing Yuan, and Zhiqiang Wei. 2020. Drug–target affinity prediction using graph neural network and contact maps. *RSC advances* 10, 35 (2020), 20701–20712.
- [9] Jeroen Kazius, Ross McGuire, and Roberta Bursi. 2005. Derivation and validation of toxicophores for mutagenicity prediction. *Journal of medicinal chemistry* 48, 1 (2005), 312–320.
- [10] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv:1412.6980* (2014).
- [11] Thomas N Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *ICLR*.
- [12] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Predict then propagate: Graph neural networks meet personalized pagerank. In *ICLR*.
- [13] Yongjiang Liang and Peixiang Zhao. 2017. Similarity Search in Graph Databases: A Multi-Layered Indexing Approach. In *ICDE*. 783–794.
- [14] Ana Lucic, Maartje A Ter Hoeve, Gabriele Tolomei, Maarten De Rijke, and Fabrizio Silvestri. 2022. CF-gnnexplainer: Counterfactual explanations for graph neural networks. In *AIStats*.
- [15] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. 2020. Parameterized explainer for graph neural network. In *NeurIPS*.
- [16] Qiaozhu Mei, Jian Guo, and Dragomir Radev. 2010. Divrank: the interplay of prestige and diversity in information networks. In *SIGKDD*.
- [17] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. 2005. Efficient computation of frequent and top-k elements in data streams. In *ICDT*.
- [18] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe W. J. Jiang, Ebrahim M. Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Sungmin Bae, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Anand Babu, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2020. Chip Placement with Deep Reinforcement Learning. *CoRR abs/2004.10746* (2020).
- [19] Dheepikaa Natarajan and Sayan Ranu. 2016. A scalable and generic framework to mine top-k representative subgraph patterns. In *ICDM*.
- [20] Robin Pemantle. 1992. Vertex-reinforced random walk. *Probability Theory and Related Fields* 92, 1 (1992), 117–136.
- [21] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. Deepwalk: Online learning of social representations. In *SIGKDD*.
- [22] Kaivalya Rawal and Himabindu Lakkaraju. 2020. Beyond individualized recourse: Interpretable and interactive summaries of actionable recourses. In *NeurIPS*.
- [23] Kaspar Riesen and Horst Bunke. 2008. IAM graph database repository for graph based pattern recognition and machine learning. In *Joint IAPR International Workshops on Statistical Techniques in Pattern Recognition (SPR) and Structural and Syntactic Pattern Recognition (SSPR)*. Springer, 287–297.
- [24] Alberto Sanfeliu and King-Sun Fu. 1983. A distance measure between attributed relational graphs for pattern recognition. *IEEE transactions on systems, man, and cybernetics* 3 (1983), 353–362.
- [25] Edoardo Sarubbi, Pier Fausto Seneci, Michael R Angelastro, Norton P Peet, Maurizio Denaro, and Khalid Islam. 1993. Peptide aldehydes as inhibitors of HIV protease. *FEBS letters* 319, 3 (1993), 253–256.
- [26] Nino Shervashidze, Pascal Schweitzer, Erik Jan Van Leeuwen, Kurt Mehlhorn, and Karsten M Borgwardt. 2011. Weisfeiler-lehman graph kernels. *JMLR* 12, 9 (2011).
- [27] Juntao Tan, Shijie Geng, Zuohui Fu, Yingqiang Ge, Shuyuan Xu, Yunqi Li, and Yongfeng Zhang. 2022. Learning and evaluating graph neural network explanations based on counterfactual and factual reasoning. In *WebConf*.
- [28] Paul Voigt and Axel Von dem Bussche. 2017. The eu general data protection regulation (gdpr). *A Practical Guide, 1st Ed., Cham: Springer International Publishing* 10, 3152676 (2017), 10–5555.
- [29] Minh Vu and My T Thai. 2020. Pgm-explainer: Probabilistic graphical model explanations for graph neural networks. In *NeurIPS*.
- [30] Nikil Wale and George Karypis. 2006. Comparison of Descriptor Spaces for Chemical Compound Retrieval and Classification. In *ICDM*.
- [31] Jiacheng Xiong, Zhaoping Xiong, Kaixian Chen, Hualiang Jiang, and Mingyue Zheng. 2021. Graph neural networks for automated de novo drug design. *Drug Discovery Today* 26, 6 (2021), 1382–1393.
- [32] Shuang-Hong Yang, Bo Long, Alex Smola, Narayanan Sadagopan, Zhaohui Zheng, and Hongyuan Zha. 2011. Like like alike: joint friendship and interest propagation in social networks. In *WebConf*.
- [33] Rex Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. Gnnexplainer: Generating explanations for graph neural networks. In *NeurIPS*.
- [34] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph Convolutional Neural Networks for Web-Scale Recommender Systems. In *SIGKDD*.
- [35] Hao Yuan, Jiliang Tang, Xia Hu, and Shuiwang Ji. 2020. XGNN: Towards Model-Level Explanations of Graph Neural Networks. In *SIGKDD*.
- [36] Tong Zhao, Gang Liu, Daheng Wang, Wenhao Yu, and Meng Jiang. 2022. Learning from Counterfactual Links for Link Prediction. In *ICML*.